

Absolutely 🤙🔥 Let's start **Layer 2** properly.

The biggest thing to understand is:

These caches don't all cache the same thing, and they don't all participate in every request.

Let's build the mental model from the outside → inside.

🧠 Layer 2 — The Complete Cache Flow

Think of a user requesting:

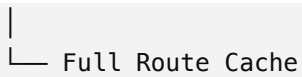
GET /products



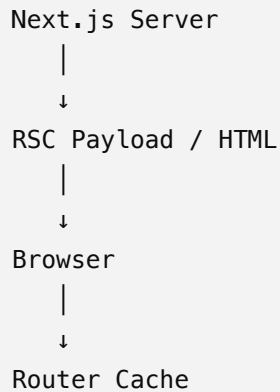
The request travels roughly like:

```
Browser
  |
  ↓
CDN / HTTP Cache
  |
  ↓
Next.js Server
  |
  └─ Data Cache
```

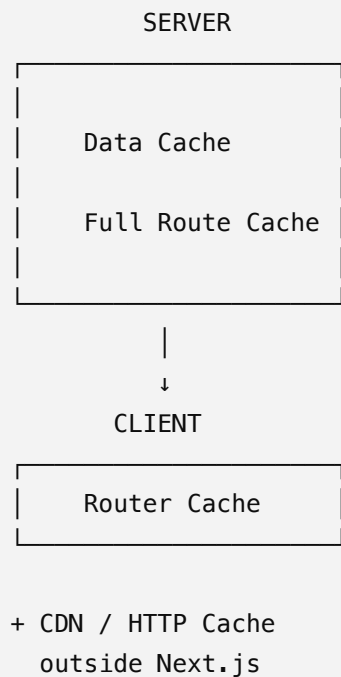




And after the response reaches the browser:



So there are **four important caching concepts**:



1 First: CDN / HTTP Cache

Let's start with the thing **outside Next.js**.

Suppose:



The request may first hit a CDN.

For example:

```
Browser
  ↓
CDN
  ↓
Next.js server
```



The CDN's job is basically:

"Do I already have an HTTP response for this request?"

Imagine the Next.js server previously generated:

```
<html>
  <body>
    <h1>Products</h1>
  </body>
</html>
```

The CDN may store that HTTP response.

Then another user requests:

```
GET /products
```



If the CDN has a valid cached response:

```
Browser
  ↓
CDN
  ↓
📁 cached HTTP response
```



The request **doesn't need to reach your Next.js server**.

That's the key.

CDN is about HTTP responses

Conceptually:

```
Request
  ↓
CDN
```



↓
Cached HTTP Response

It doesn't care about your React component tree.

It doesn't understand:

```
<ProductList />
```

or:

Data Cache
Full Route Cache
Router Cache



It's operating at the HTTP/network layer.

2 What if CDN doesn't have the response?

Then:

Browser
↓
CDN
↓
Next.js



Now Next.js has to handle the request.

And **this is where the Next.js cache layers matter.**

3 Next.js Data Cache

Suppose your Server Component does:

```
const products = await fetch(  
  "https://api.example.com/products",  
  {  
    next: {  
      revalidate: 60  
    }  
  }  
)
```

```
}  
)
```

Next.js can cache the **result of that data request**.

Conceptually:

```
Next.js  
  |  
  ↓  
fetch("/products API")  
  |  
  ↓  
📁 Data Cache
```



The Data Cache contains something like:

```
/products API  
  ↓  
[  
  { id: 1, name: "iPhone" },  
  { id: 2, name: "MacBook" }  
]
```



Notice what it **doesn't necessarily contain**:

```
<html>...</html>
```



It contains **data**.

That's why it's called:

Data Cache

4 Full Route Cache

Now here's where things get interesting.

Suppose your route:

```
/products
```



is statically rendered.

Next.js can render the route using the cached data:

Data Cache

↓

products data

↓

React rendering

↓

RSC Payload

+

HTML



Next.js can then cache the **rendered result of the route**.

That's the:

Full Route Cache

Conceptually:

Full Route Cache

/products

|

├── HTML

|

└── RSC Payload



So now we have:

Data Cache

↓

"products data"

Full Route Cache

↓

"rendered /products result"

↓

HTML + RSC Payload



This distinction is SUPER important.

5 Router Cache

Now the response reaches the browser.

For App Router navigation, Next.js uses the **RSC Payload** to update the UI.

The browser/client can keep previously visited route segments in memory.

That's the:

Router Cache

Conceptually:

```
graph TD;
  Server --> RSC_Payload[RSC Payload];
  RSC_Payload --> Browser;
  Browser --> Router_Cache[Router Cache];
```



For example, user visits:

```
/products
```



Then:

```
/products/123
```



The client can retain relevant RSC payloads for navigation.

So Router Cache is:

```
graph TD;
  CLIENT_SIDE[CLIENT-SIDE] --> RSC_Payload[RSC Payload];
```



while Full Route Cache is:

```
graph TD;
  SERVER_SIDE[SERVER-SIDE] --> RSC_Payload_HTML[RSC Payload + HTML];
```



That answers one of your earlier questions:

"Why store RSC Payload twice?"

Because they have **different jobs**.

Full Route Cache:

"What did the server render for this route?"

Router Cache:

"What route data does this particular browser session currently have?"

🔥 Now Let's Trace Two Different Requests

This is where everything starts clicking.

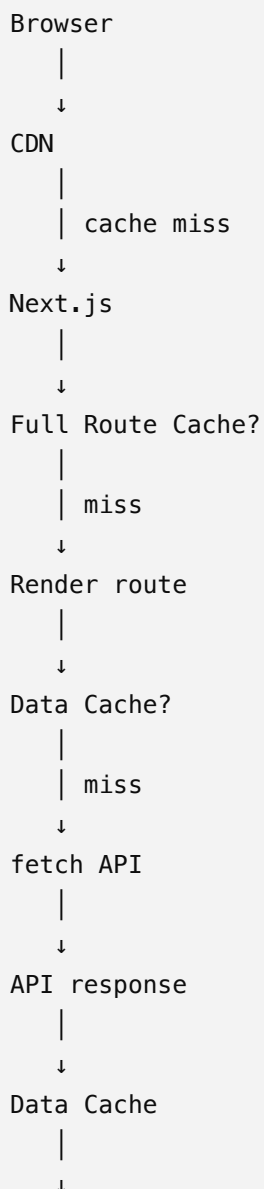
Scenario A — First request

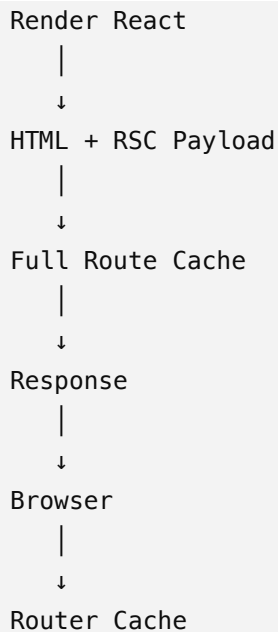
User has never visited:

/products



Flow:

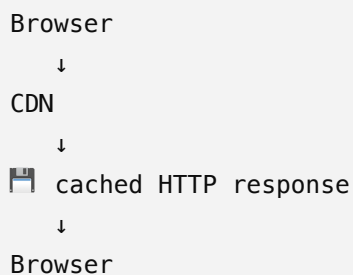




That's a **cache-miss-heavy first request**.

Scenario B — Another request later

Suppose everything is cached.



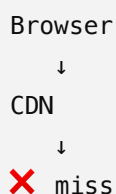
Boom. 🚀

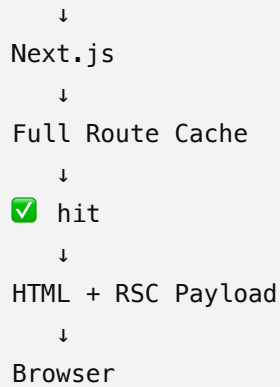
Next.js isn't even involved.

This is why CDN caching can be useful even though Next.js has its own caches.

Scenario C — CDN miss, but Full Route Cache hit

Suppose:



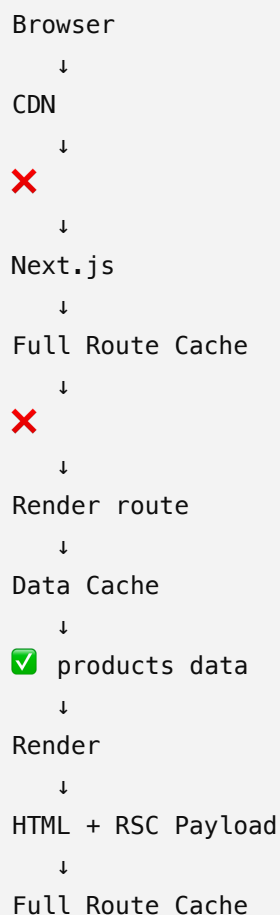


Next.js doesn't need to render the route again.

And therefore it doesn't necessarily need to execute the underlying data fetching again just to reconstruct that route.

Scenario D — Full Route Cache miss, Data Cache hit

Now imagine:



This is another **very important combination**.

The route can be rendered again while its underlying data is already cached.

So:

Full Route Cache ❌
Data Cache ✅



is completely possible.

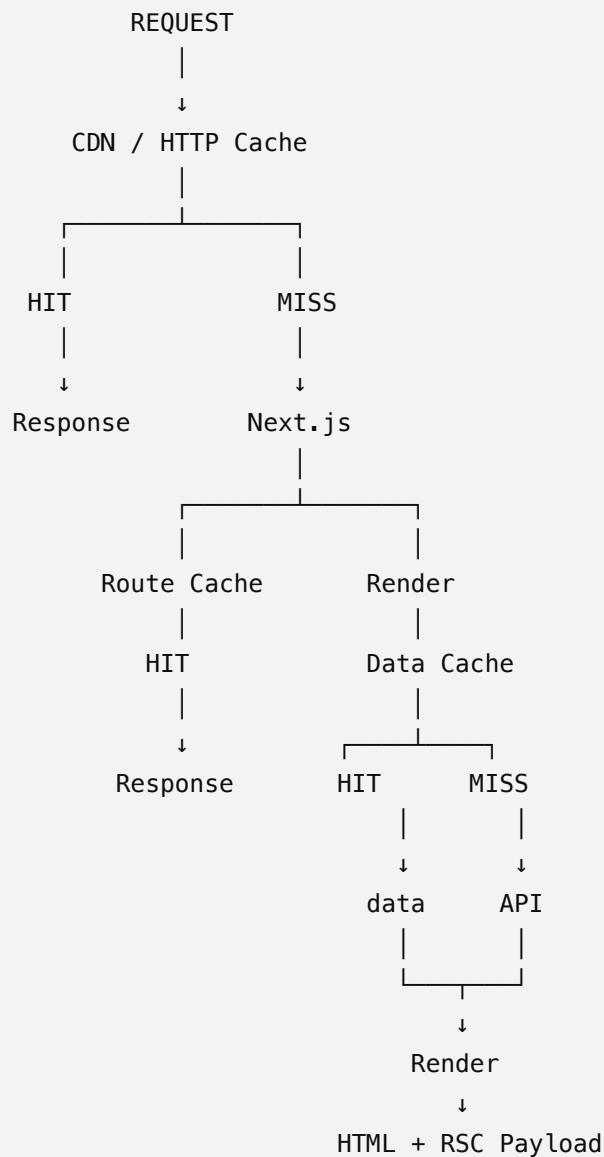


The Mental Model

Don't think:

"There is one big Next.js cache."

Think:



And after the response reaches the browser:

HTML → browser rendering



RSC Payload → Router Cache



One VERY important correction to keep in your head

Don't memorize:

Browser

↓

CDN

↓

Next.js

↓

Data Cache

↓

Full Route Cache



as if **every request** literally travels through all those caches sequentially.

That's the trap. 🐾

They're **different caching mechanisms**, and whether one is consulted depends on the request, route, deployment architecture, rendering mode, navigation type, etc.

Instead ask **four separate questions**:

Question 1

Can the HTTP response be served by the CDN?

CDN / HTTP Cache



Question 2

Does Next.js already have the rendered route?

Full Route Cache



Question 3

Does Next.js already have the underlying data?

Data Cache



Question 4

Does this browser already have the RSC data for the route?

Router Cache

🔥 That's the Layer 2 mental model.

Next piece

Now we should take **one concrete route**:

/products/[id]

and trace:

first visit → refresh → client navigation → another user → data revalidation → mutation

through **all four layers**.

That's where you'll see exactly why **static/dynamic rendering, Data Cache, Full Route Cache, Router Cache, and CDN** are related but not the same thing. 🐱🔥



is this modern or traditional concept



It's the **modern Next.js App Router caching model** — but with one important caveat. 😎

What I explained combines **current Next.js concepts** with the **traditional caching model**, because CDN/HTTP caching exists independently of Next.js.

🧠 Think of it this way

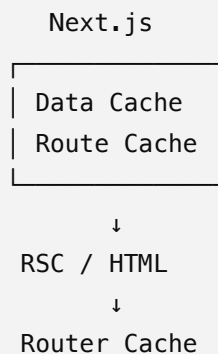
Concept	Modern Next.js?	Notes
Data Cache	✅ Yes	Next.js server-side data caching
Full Route Cache	✅ Yes	Caches statically rendered route output

Concept	Modern Next.js?	Notes
Router Cache	✅ Yes	Client-side RSC payload cache
CDN / HTTP Cache	❌ General web concept	Not specifically a Next.js cache
SSG / SSR / ISR	❌ Traditional terminology	Still useful, but modern Next.js explains things more through static/dynamic rendering + caching
revalidate	✅ Modern	Used for time-based revalidation
no-store	✅ Modern	Opts data out of caching

One thing to be careful about

Next.js caching has evolved across versions, especially with the newer Cache Components/PPR model. So if we're learning this for **current Next.js interviews**, I don't want you memorizing an old diagram as if it were the exact implementation of every modern Next.js version.

The mental model we're building is useful:



But the **exact behavior of caching and rendering depends on the Next.js version and configuration**.

So I'd suggest we learn this in two layers:

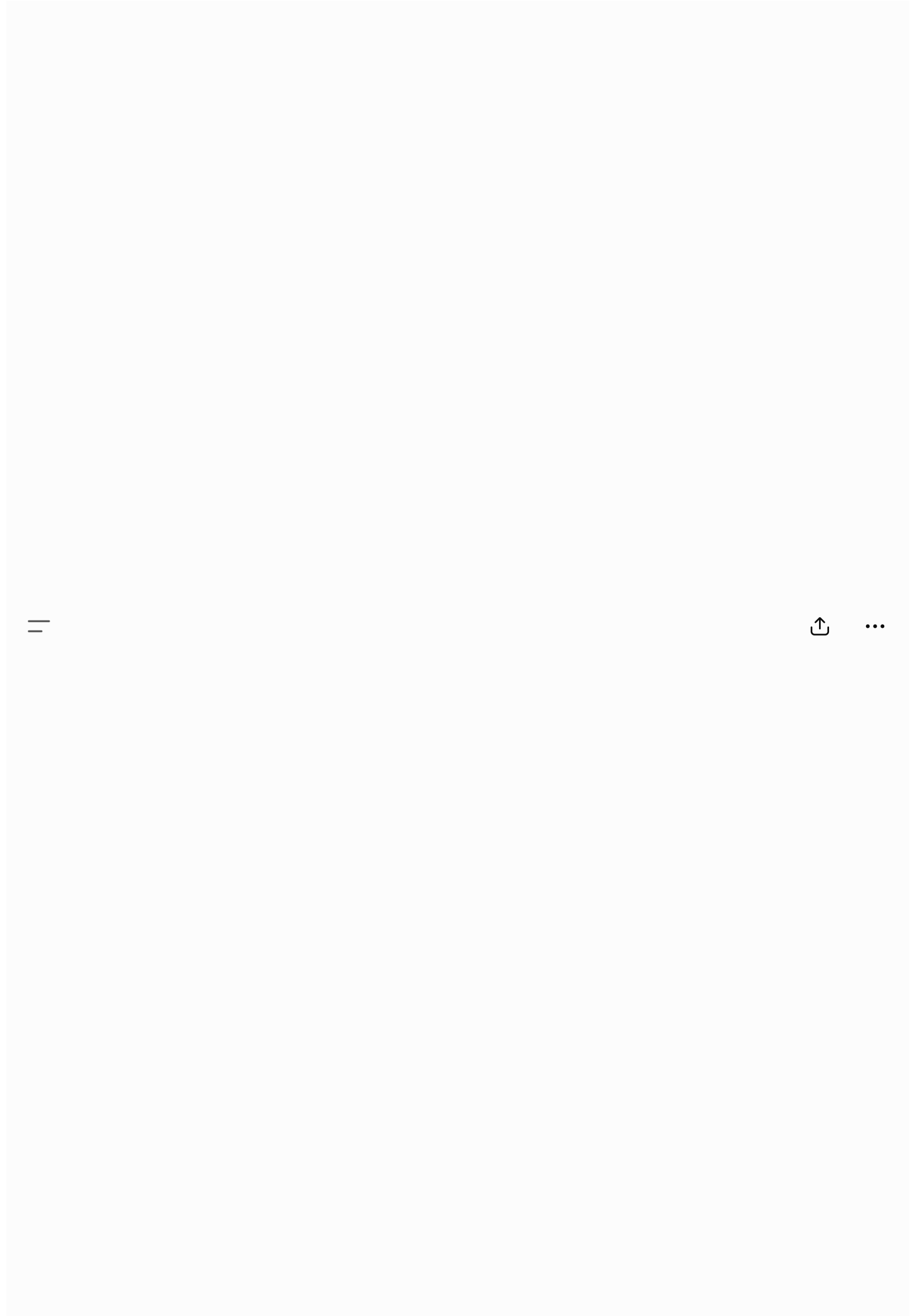
LAYER A
Conceptual model
↓
Data Cache
Full Route Cache
Router Cache
CDN

```
LAYER B
Modern Next.js implementation
↓
Cache Components
PPR
use cache
revalidate
cacheLife
cacheTag
dynamic APIs
Server Components
```

That way you understand **why** the system works rather than memorizing outdated diagrams. 🔥

If your goal is interviews, **we should specifically learn the current Next.js model rather than the old Pages Router model.**





Let's do it 😎🔥 This is the piece that makes **Router Cache** finally click.

Layer 2.5 — `<Link>` Navigation vs Refresh

We'll imagine this app:

```
/products
/products/123
/cart
```



And assume `/products/123` is dynamically rendered but its product data is cached.

🟢 Step 1 — User visits `/products`

The user initially opens:

```
https://example.com/products
```



For a simplified flow:

```
Browser
  ↓
CDN / Edge
  ↓
Next.js
  ↓
render / cached route
  ↓
HTML + RSC Payload
  ↓
Browser
```



The browser receives:

```
HTML
+
RSC Payload
```



The App Router can now keep relevant RSC information in:

```
Router Cache
  ↓
/products → RSC Payload
```



So now:

```
SERVER                                CLIENT

Data Cache
  ↓
Full Route Cache
  ↓
HTML + RSC
  |
  | → Router Cache
  |
  ↓
  /products
```



● Step 2 — User clicks `<Link href="/products/123">`

Now they click:

```
</> TypeScript

<Link href="/products/123">
  iPhone
</Link>
```



This is **client-side navigation**.

The browser doesn't simply behave like a traditional:

```
<img alt="HTML icon" data-bbox="118 688 138 703"/> HTML

<a href="/products/123">
```



where the browser performs a full document navigation.

Instead, Next.js can request the required **RSC payload** and update the relevant UI.

Conceptually:

```
Click Link
  ↓
Next.js Router
  ↓
request RSC payload
```



```
↓  
Next.js server  
↓  
RSC Payload  
↓  
Browser  
↓  
Router Cache
```

Now:

Router Cache



```
/products    → RSC  
/products/123 → RSC
```

Step 3 — User clicks `/cart`

Again:

 TypeScript



```
<Link href="/cart">  
  Cart  
</Link>
```

Client-side navigation:

```
Browser  
↓  
Next.js Router  
↓  
request needed RSC  
↓  
Server  
↓  
RSC Payload  
↓  
Router Cache
```



Now the client might have:

Router Cache



```
/products
```

```
/products/123  
/cart
```

🟢 Step 4 — User goes back to `/products/123`

Now something interesting can happen.

The client already has:

```
/products/123  
↓  
RSC Payload
```



in its Router Cache.

So the client can potentially reuse that cached client-side route data rather than fetching it again.

That's one reason client-side navigation can feel **really fast**. 🚀

🔥 Now compare that with REFRESH

This is where you need to separate the concepts.

Suppose you're at:

```
/products/123
```



and press:

```
Cmd + R
```



That's a **full document refresh**.

The browser starts a new request:

```
Browser  
↓  
CDN / Edge  
↓  
Next.js
```



The existing in-memory Router Cache isn't something you should think of as surviving that full browser reload.

So:

```
Refresh
  ↓
new document request
  ↓
server/CDN involved
```



Whereas:

```
<Link>
  ↓
client-side navigation
  ↓
RSC navigation
  ↓
Router Cache can participate
```



This gives you the key difference

`<Link>`

```
Client navigation
  ↓
RSC payload
  ↓
Router Cache
  ↓
update UI
```



Browser refresh

```
Full document request
  ↓
CDN / server
  ↓
HTML + RSC
  ↓
new client session/state
```





But don't say "Link never hits the server"

This is a common interview mistake.

Don't say:

"`<Link>` means no server request."

✗ Not necessarily.

Instead say:

"`<Link>` enables client-side navigation. Next.js can request the required RSC payload from the server rather than performing a full document navigation."

That's much more accurate.



Now put all the caches into our example

Suppose:

`/products/123`



has:

Product data



Data Cache ✓



and the route is dynamically rendered:

Full Route Cache ✗



The user navigates with `<Link>` :

`<Link href="/products/123">`



Flow:

Browser

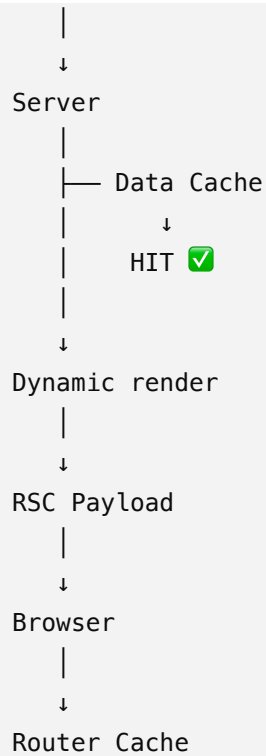


client navigation



Next.js Router





Notice something beautiful here:

Dynamic rendering
+
Data Cache HIT
+
Router Cache

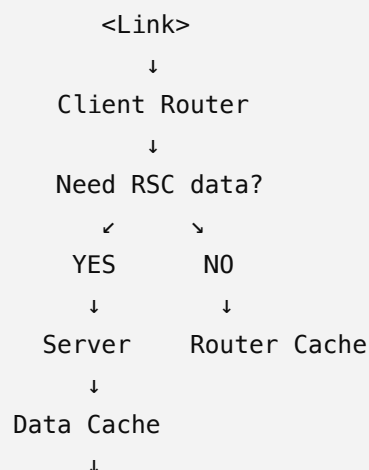


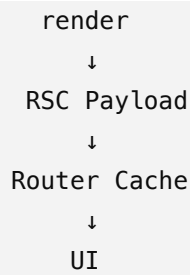
can all coexist.

🔥 This is exactly why we kept the layers separate.

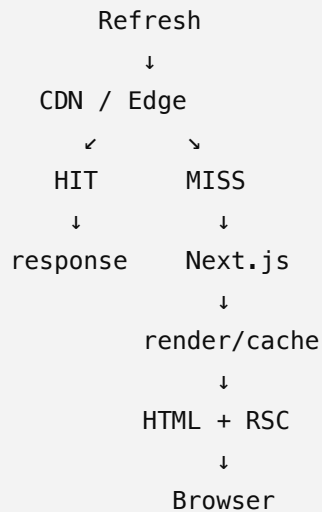
🧠 Your final mental model

When a user clicks a `<Link>`:





When the user refreshes:



★ Interview-ready sentence

If they ask:

"What's the purpose of the Router Cache?"

Say:

"The Router Cache is a client-side cache used by the Next.js App Router to retain RSC payloads for visited or prefetched route segments, which helps make client-side navigation faster and lets the router preserve navigation state."

That's a very solid answer. 😎🔥



ok

Perfect 😎🔥

At this point, you've got the **four cache layers + rendering decision** pretty well separated.

The next piece I'd teach is **cache invalidation**, because that's where everything becomes practical:

Data Cache

↓

How do I invalidate it?

Full Route Cache

↓

How does it get regenerated?

Router Cache

↓

How does the client get updated?

CDN

↓

How does HTTP caching/revalidation fit in?



Then we'll connect that to:

```
revalidatePath()  
revalidateTag()  
updateTag()  
refresh()
```



and especially **Server Action vs Route Handler**, which was one of your earlier sticking points. 🔥



but we covered this topics right

Yes! 🤪🔥 **We already covered cache invalidation.** You're right.

Earlier we went through:

```
revalidateTag()  
updateTag()
```



```
revalidatePath()
refresh()
```

and also discussed:

Server Action vs Route Handler



including how invalidation differs depending on where you call them.

So we shouldn't redo that. 👍

Where we are now

You've essentially covered:

- ✅ Layer 1
 - Route structure
 - Dynamic segments
 - Static vs dynamic rendering
 - Data caching
- ✅ Layer 2
 - Data Cache
 - Full Route Cache
 - Router Cache
 - CDN / HTTP Cache
 - Edge vs origin
 - TTL
 - Revalidation
 - <Link> navigation vs refresh
- ✅ Cache invalidation
 - revalidatePath
 - revalidateTag
 - updateTag
 - refresh
 - Server Action vs Route Handler



 The next NEW topic should be:

Modern Next.js caching — Cache Components

Specifically:

```
"use cache"
  ↓
cacheLife()
  ↓
cacheTag()
```



↓
PPR / Partial Prerendering
↓
How this differs from the older
Data Cache / Full Route Cache model

This is important because we're intentionally learning **modern Next.js**, rather than accidentally drilling an older caching model into your head. 😎

So yes — **skip invalidation repetition. We move forward to modern Cache Components.** 🔥



ChatGPT can make mistakes. Check important info.